System Testing: Testing the Whole System from the Outside

SE 413/513 — Software Testing

Course Reading Material

Unit: Test Levels

© Dr. Kamrul Hasan, Grand Valley State University. All rights reserved.
For permissions or questions regarding the use of this material, contact hasanka@gvsu.edu.

Please cite [1] when using, adapting, or reproducing this material. The corresponding BibTeX entry is provided below:

```
@misc{hasan2026stesting,
author = {Hasan, Kamrul},
year = {2026},
title = {Software Testing},
institution = {Grand Valley State University},
url = {https://github.com/mdkamrulhasan/software-testing}
}
```

Contents

1 Learning Objectives **3**

2 Where We Are **3**

3 What Is System Testing? **4**
3.1 Definition . . . 4
3.2 Where It Sits Among the Test Levels . . . 4
3.3 Verification, Not Yet Validation . . . 4

4 The System Under Test **4**
4.1 The Shop Grows Up . . . 4
4.2 The System Requirements Specification . . . 5
4.3 Functional and Non-Functional Requirements . . . 6

5 Planning System Tests **7**

6 The Test Environment and Harness **8**
6.1 Starting the System as a Process . . . 8
6.2 Observe from the Outside . . . 8

7 Functional System Testing: Scenarios **9**
7.1 From Use Cases to Scenarios 9

8 Non-Functional System Testing **10**
8.1 Concurrency: Many Customers at Once 10
8.2 Performance 12
8.3 Security and Robustness 13
8.4 Recoverability and Operability 14
8.5 Other System Test Types 15

9 System Tests in the Suite **15**
9.1 Markers and Run Time 15
9.2 One Defect, Four Levels 15

10 Common System-Level Failures **16**

11 Putting It Together: A Hands-On Walkthrough **16**

12 Best Practices **17**

13 Common Mistakes **17**

14 Applying This to Your Semester Project **18**

15 A Practical Mental Model **19**

16 Summary **19**

17 Practice Exercises **20**

18 Discussion Questions **20**

19 Key Takeaways for SE 413/513 **21**

20 References **21**

1 Learning Objectives

After completing this reading, you should be able to:

- Define system testing, and distinguish it from integration testing (Week 10) and acceptance testing (this week's companion reading) by its question, test basis, and environment.
- Read a system requirements specification, separate its **functional** from its **non-functional** requirements, and relate the latter to the ISO/IEC 25010 quality characteristics.
- Write the essential parts of a **system test plan**: scope, test basis, environment, approach, entry and exit criteria, traceability, and risks.
- Build a **system test harness** in `pytest` that starts the system as a separate process and tests it only through its external interfaces.
- Design functional system tests from **use-case scenarios**, and non-functional system tests for **concurrency**, **performance**, **security and robustness**, and **recoverability**.
- Explain why some defects can be found *only* at the system level, and give an example.

2 Where We Are

Week 10 assembled the shop from real parts — a Flask API, an `OrderService`, a repository over SQLite, and a client for the PayFast payment provider — and tested each integration point between them. By the end of that week, every one of the six integration points had tests, and those tests found a data-format defect (dollars sent where cents were expected) and a transaction defect (stock committed without its order) that mocks could not see.

Week 10's most complete tests, in `test_api.py`, already sent HTTP requests to the fully assembled shop. So what is left? Three things those tests never did:

1. **They never ran the shop.** Flask's `test_client()` calls the application inside the test process. Nothing listened on a port; nothing was started, configured, stopped, or restarted.
2. **They had one customer at a time.** The test client handles requests one after another on a single thread. A real shop serves many customers at once.
3. **They could reach inside.** The `stock` fixture read the database directly, and tests set `provider.config` on the fake PayFast object. A customer can do neither.

System testing removes all three. This reading's demo runs the shop *as a separate process*, the way it would be deployed, and tests it the way the outside world meets it: over HTTP, with many clients, without looking in.

Key Idea

Integration testing asks, "Do these components work *together*?" System testing asks, "Does the *whole system*, running in an environment like production, do what its specification says — including how fast, how safely, and how reliably?"

3 What Is System Testing?

3.1 Definition

System testing is the test level that focuses on the overall behavior and capabilities of a whole system or product[2]. It typically includes **functional testing of end-to-end tasks** and **non-functional testing of quality characteristics** such as performance, security, and reliability. Its test basis is the **system specification**: system requirements, use cases, and quality requirements. It is run in a test environment that should correspond, as closely as practical, to the production environment, and it is often carried out by testers who did not write the code[2, 3].

3.2 Where It Sits Among the Test Levels

Level	Question	Test basis	In this course's shop
Unit	Does this unit compute the right thing?	Detailed design, docstrings	`tests/unit/`: `price_order()` alone
Integration (Wk 10)	Do these components agree at their interfaces?	Architecture, API contracts	`tests/integration/`: assembled *inside* the test process
System	**Does the whole system meet its specification?**	**System requirements**	**`tests/system/`: a running server, HTTP only**
Acceptance	Is it the system the users need?	User stories, business needs	`tests/acceptance/`: Gherkin scenarios (companion reading)

Two words in the system row matter most. **Whole**: every component is the real one, configured as it would be deployed, except for third parties the team does not control (here, PayFast). **Specification**: system tests are *black-box*. They are derived from what the system is supposed to do, using the functional test design techniques of Weeks 3 and 8, and they check only what can be observed from outside.

3.3 Verification, Not Yet Validation

System testing checks the system against its specification: it answers "did we build the system right?" — **verification**. Whether the specification describes what users actually need — "did we build the right system?", **validation** — is the subject of acceptance testing[4]. A system can pass every system test and still be the wrong system. The companion reading shows exactly that happening.

4 The System Under Test

4.1 The Shop Grows Up

This week's demo, `demos/system_acceptance_testing/`, starts from Week 10's shop and adds what a system that runs on its own needs:

- `GET /health`, a health check that needs no API key, for load balancers and monitoring;
- `GET /products`, a product catalog with prices and stock;

- coupon codes on `POST /orders` (designed in the companion reading);
- a `serve.py` that runs the shop and the fake PayFast as real servers, keeps the database between restarts, and prints one `READY` line when both are listening.

Week 10's folder is unchanged; its reading quotes it line by line. Figure 1 shows the system and its test environment.

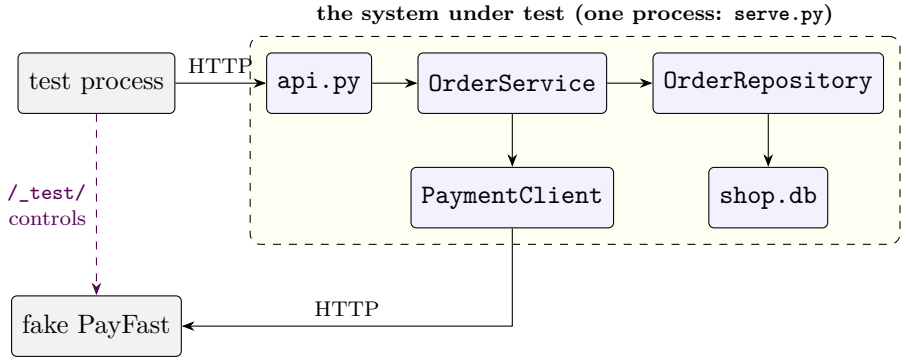


Figure 1: The shop as a system. The dashed box runs as one process; the test process and the fake PayFast are the test environment around it. Solid arrows are what production also has; the dashed arrow exists only in testing.

The dashed box is the system under test. The test process talks to it only through its HTTP interface. The fake PayFast is part of the *test environment*: it stands in for a third party, and it offers test-only control endpoints (`POST /_test/config` to make it slow, decline cards, or go down; `GET /_test/charges` to see what it actually charged), much as a real provider's sandbox does.

4.2 The System Requirements Specification

The test basis is `specs/system_requirements.md`. Excerpt:

Shop system requirements (excerpt)

SYS-1 A customer can list the products, with each product's name, price, and current stock.
SYS-2 A customer can order 1 to 100 units of one product. The shop charges the order total through PayFast and records the order as `paid`.
SYS-3 A customer can fetch a placed order by its id.
SYS-4 The shop never sells stock it does not have, *even when many customers order at the same time.*
SYS-5 If an order cannot be paid, the customer is not charged, the stock is returned, and the response says why.
SYS-6 Every request except `GET /health` requires the shop's API key.
SYS-7 Every response, including every error, is JSON. The shop never returns an HTML page, a stack trace, or other internal details, whatever the request contains.
SYS-8 Orders and stock survive a restart of the shop.

NFR-1 (Performance) With 10 customers ordering at the same moment, and PayFast answering each charge in 100 ms, every customer gets a response within 1 second.
NFR-2 (Recoverability) When PayFast recovers from an outage, the shop accepts orders again without being restarted.
NFR-3 (Operability) `GET /health` answers `200` within 1 second, needs no API key, and stays `200` during a PayFast outage.

Notice what the specification does *not* mention: classes, tables, SQL, threads. It describes the system as its users, operators, and attackers meet it. That is what makes it a system test basis.

4.3 Functional and Non-Functional Requirements

SYS-1 to SYS-8 say *what* the system does: they are **functional** requirements. NFR-1 to NFR-3 say *how well* it does it: they are **non-functional**, or **quality**, requirements. The standard vocabulary for quality is the product quality model of ISO/IEC 25010, whose 2023 edition has nine characteristics[6]:

Characteristic	Typical system test	In this reading
Functional suitability	End-to-end scenarios against requirements	Section 7
Performance efficiency	Response time and throughput under load	NFR-1, Section 8.2
Compatibility	Coexistence and interoperability with other systems	(PayFast: Week 10)
Interaction capability*	Usability: can real users accomplish their tasks?	—
Reliability	Fault tolerance, recoverability, availability	NFR-2, SYS-8, Section 8.4
Security	Access control, input handling, confidentiality	SYS-6, SYS-7, Section 8.3
Maintainability	(mostly reviews and static analysis, not system tests)	—
Flexibility*	Installability, adaptability to other environments	`serve.py` options
Safety*	Avoiding harm to people or the environment	—

*Renamed or new in the 2023 edition: *usability* became *interaction capability*, *portability* became *flexibility*, and *safety* was added.

A quality requirement is only testable if it is **measurable**. "The shop should be fast" cannot fail a test; NFR-1 can, because it names a load (10 simultaneous customers), a condition (PayFast at 100 ms), and a threshold (1 second).

Common Pitfall

Non-functional requirements that nobody wrote down. If the specification says nothing about concurrency, performance, or hostile input, the system will still *have* behavior there — just untested behavior. A system tester's first job is often to ask for the missing requirements, as Section 8.3 shows.

5 Planning System Tests

Unit and integration tests are usually written alongside the code. System testing is a larger, separate activity with its own environment and its own schedule, so it is planned. A **test plan** records the decisions[7, 2]. The demo's one-page plan, `specs/system_test_plan.md`, has the essential sections:

Section	For the shop
Scope	In: the shop as deployed by `serve.py`. Out: PayFast itself; user interfaces; load beyond 10 customers.
Test basis	`system_requirements.md`, `shop_api.md`
Environment	One process per test; a fresh SQLite file; the fake PayFast over real HTTP; real threads and sockets.
Approach	Which test types, which techniques, which files (Sections 7–8.4).
Entry criteria	All unit and integration tests pass; `serve.py` starts; `/health` answers `200`.
Exit criteria	Every requirement has a passing system test; no open high-severity finding.
Traceability	Requirement → system tests (excerpt below).
Risks	The fake PayFast behaves better than the real one; timing tests can be flaky; a laptop is not production hardware.

Entry criteria stop system testing from starting on a build that is not ready: if the integration tests fail, system test failures will mostly rediscover them. **Exit criteria** define "done" before testing starts, so that the decision to release is not made by whoever is most tired.

Its traceability section maps every requirement to at least one system test; open `specs/system_test_plan.md` to see the full matrix.

Tip

Scale the plan to the system. For a class project, one page in the repository is right; for a regulated product, ISO/IEC/IEEE 29119-3 lists the full set of test documentation[7]. What matters is that the decisions — what is in scope, where you test, when you stop — are made deliberately and written down.

6 The Test Environment and Harness

6.1 Starting the System as a Process

A system test must test the system *as it runs*. The harness in `tests/conftest.py` starts `serve.py` with `subprocess`, on ports the operating system chooses (`--port 0`), with a fresh database file, and waits for its `READY` line:

```
def _start(tmp_path, db_path, seed=None, extra_args=()):
    """Run serve.py and wait for its READY line."""
    args = [sys.executable, "serve.py", "--port", "0", "--db", str(db_path), *extra_args]
    ...
    process = subprocess.Popen(args, cwd=DEMO_DIR, stdout=subprocess.PIPE,
                               stderr=log, text=True)
    line = ...  # the first line of stdout, or a 10-second timeout
    if not line.startswith("READY"):
        process.kill()
        pytest.fail(f"shop did not start; see {log_path}:\n{log_path.read_text()}")
    fields = dict(part.split("=", 1) for part in line.split()[1:])
    shop = RunningShop(process, fields["shop"], fields["provider"], db_path, log_path)
    assert shop.get("/health").status_code == 200  # ready means healthy
    return shop


@pytest.fixture
def shop(start_shop):
    """The shop, running, with the default catalog:
    MUG $12.50 x5, STICKER $3.00 x100, TEE $20.00 x0."""
    return start_shop()
```

Three details carry over from Week 10's lessons. Port 0 lets tests run in parallel without colliding. A fresh database file per test means no test depends on another's leftovers. And the server's own log goes to a file in `tmp_path`, which the failure message names — when a system test fails, the server log is usually the first thing you need.

6.2 Observe from the Outside

Compare how Week 10 and this week check the same fact, "the stock went down by two":

Integration test (Week 10)	**System test (this week)**
`stock("MUG")`: a second SQLite connection reads `products`	`shop.stock("MUG")`: `GET /products`, as any client could
`provider.config["MODE"] = "decline"`	`shop.provider(mode="decline")`: an HTTP call to the test environment
`provider.charges` (a Python list)	`shop.charges()`: `GET /_test/charges`

Common Pitfall

Reaching into the system under test. Opening the database from a system test, importing the system's modules, or patching its functions makes the test depend on internals — and quietly stops it from being a system test. If a requirement cannot be checked from the outside, that is itself a finding: either the requirement is not externally observable (and belongs at a lower level), or the system lacks an interface it needs (an operator report, an audit endpoint). Controlling the *test environment* — the fake PayFast — is fine; it is not part of the system.

7 Functional System Testing: Scenarios

7.1 From Use Cases to Scenarios

Functional system tests are usually organized as **end-to-end scenarios**: sequences of steps a user performs to reach a goal[3, 2]. A **use case** describes one goal, with a **main success scenario** and **alternative flows** for what can go differently. For "place an order":

Flow	What happens	System test
Main success	browse, order, look the order up	`test_browse_order_and_look_the_order_up`
Alt.: sold out	refused, nothing charged	`test_sold_out_product_is_refused_...`
Alt.: card declined	refused, stock back, order on record	`test_declined_card_charges_nothing_...`
Alt.: stock runs out	later customers refused	`test_several_customers_one_after_another`

```
pytestmark = pytest.mark.system

def test_browse_order_and_look_the_order_up(shop):
    """SYS-1, SYS-2, SYS-3: the main success scenario."""
    catalog = shop.get("/products").json()
    mug = next(p for p in catalog if p["sku"] == "MUG")
    assert (mug["price_cents"], mug["stock"]) == (1250, 5)

    placed = shop.order("MUG", 2)
    assert placed.status_code == 201
    order = placed.json()
    assert (order["status"], order["total_cents"]) == ("paid", 2500)

    assert shop.get(placed.headers["Location"]).json() == order
    assert shop.stock("MUG") == 3
    assert [c["amount"] for c in shop.charges()] == [2500]


def test_declined_card_charges_nothing_and_returns_the_stock(shop):
    """SYS-5: and the failed order is still on record."""
    shop.provider(mode="decline")
    response = shop.order("MUG", 2)

    assert response.status_code == 402
```

```
    assert shop.stock("MUG") == 5
    assert shop.charges() == []
    assert shop.get("/orders/1").json()["status"] == "payment_failed"
```

A scenario test checks *every* externally visible consequence of the scenario, not just the response: the catalog, the order record, and what the customer was actually charged. Week 10 called this "checking both sides"; at the system level, both sides must be observed from outside.

Tip

Put the requirement IDs in the docstring (or the test name). With `pytest -v`, the test report then reads as a traceability report.

8 Non-Functional System Testing

8.1 Concurrency: Many Customers at Once

SYS-4 says the shop never oversells, "even when many customers order at the same time". The test sends ten orders for the five mugs at the same moment — one thread per customer, released together by a `threading.Barrier` — while PayFast takes 0.2 s per charge, as on a slow day:

```
def order_all_at_once(shop, bodies):
    """Send every order at (nearly) the same moment, one thread per
    customer. Returns [(status_code, seconds), ...] in input order."""
    start_line = threading.Barrier(len(bodies))

    def customer(body):
        start_line.wait()                          # everyone starts together
        started = time.perf_counter()
        response = requests.post(f"{shop.url}/orders", json=body, timeout=10,
                                 headers={"X-API-Key": "shop-key"})
        return response.status_code, time.perf_counter() - started

    with ThreadPoolExecutor(max_workers=len(bodies)) as pool:
        return list(pool.map(customer, bodies))


def test_ten_customers_race_for_the_last_five_mugs(shop):
    """SYS-4: exactly 5 orders succeed, 5 are refused, nothing is oversold."""
    shop.provider(delay=0.2)
    bodies = [{"sku": "MUG", "quantity": 1, "customer_email": f"c{i}@example.com"}
              for i in range(10)]
    statuses = [status for status, _ in order_all_at_once(shop, bodies)]

    sold, refused = statuses.count(201), statuses.count(409)
    outcome = {"sold": sold, "refused": refused,
               "stock_left": shop.stock("MUG"), "cards_charged": len(shop.charges())}
    assert outcome == {"sold": 5, "refused": 5, "stock_left": 0, "cards_charged": 5}
```

It passes. Now consider a design change a developer might reasonably propose: "Don't hold stock for payments that fail. Check the stock, charge the card, and *then* take the stock." That version is in `shop/service_racy.py`:

```
        if product["stock"] < quantity:                    # check...
            raise OutOfStock(sku)
        ...
        charge_id = self.payments.charge(total, reference=f"order-{order_id}")
        ...
        with self.connection:                               # ...then act, on a stale value
            self.repository.set_stock(sku, product["stock"] - quantity)
            self.repository.update_order(order_id, "paid", charge_id)
```

The environment variable `SHOP_SERVICE=racy` switches it in — in the test process and, because child processes inherit the environment, in the server too. Run every level:

```
$ SHOP_SERVICE=racy pytest tests/unit tests/integration -q
21 passed in 0.47s
$ SHOP_SERVICE=racy pytest tests/system/test_concurrency_and_performance.py --tb=short
E   AssertionError: assert {'sold': 10, ..._charged': 10} == {'sold': 5, '...s_charged': 5}
E     Differing items:
E     {'stock_left': 4} != {'stock_left': 0}
E     {'cards_charged': 10} != {'cards_charged': 5}
E     {'refused': 0} != {'refused': 5}
E     {'sold': 10} != {'sold': 5}
1 failed, 1 passed in 1.08s
```

All ten customers were sold a mug and charged for it; the shop had five. And it still believes it has **four** left. Every request read "stock: 5" before any charge finished, and every request then wrote back "5 minus 1". Ten writes of the same value: a **lost update**, caused by a **check-then-act** sequence with a slow step in the middle (Figure 2).

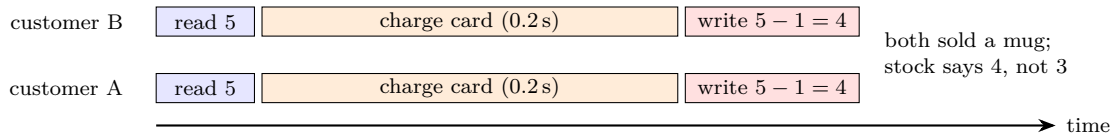


Figure 2: A lost update. Both customers read the stock before either charge finishes, and both write back the value they computed from it. With ten customers, ten mugs are sold and the stock reads 4.

Key Idea

Some defects exist only when the whole system runs *as it runs in production*: here, a multi-threaded server, separate database connections per request, and a slow external service. Integration tests that drive one request at a time through an in-process client cannot see them, however thorough they are. The correct service does not have this defect because it reserves stock with one atomic statement — `UPDATE ... SET stock = stock - ? WHERE ... AND stock >= ?` — *before* it charges (Week 10, R3/R4).

Common Pitfall

Concurrency tests that pass by luck. Without the barrier and the 0.2 s delay, the ten requests might happen not to overlap, and the racy version would sometimes pass. A concurrency test must *force* the overlap it is looking for. We ran each version eight times: the correct shop passed 8/8 and the racy shop failed 8/8.

8.2 Performance

NFR-1 is a **response-time** requirement under a stated load. The same helper measures each customer's latency:

```
def test_ten_simultaneous_orders_each_answered_within_a_second(shop):
    """NFR-1: 10 customers at once, PayFast answering in 100 ms."""
    shop.provider(delay=0.1)
    bodies = [{"sku": "STICKER", "quantity": 1, "customer_email": f"c{i}@example.com"}
              for i in range(10)]
    results = order_all_at_once(shop, bodies)
    seconds = sorted(t for _, t in results)

    print(f"\nlatency: median {statistics.median(seconds):.3f} s, "
          f"max {seconds[-1]:.3f} s")
    assert all(status == 201 for status, _ in results)
    assert seconds[-1] < 1.0
```

```
$ pytest tests/system/test_concurrency_and_performance.py -k within_a_second -s -q
latency: median 0.145 s, max 0.251 s
1 passed in 0.64s
```

The exact numbers will differ on your machine; the assertion has a wide margin (1 s against about 0.25 s) so that it fails for a real regression, not for a busy laptop. This is a **performance smoke test**: it catches an order that suddenly takes seconds (a missing timeout, a lock held across the charge), not a gradual slowdown. Real performance testing has several forms[2, 8]:

Type	Question
Load	Does it meet its targets at the expected load?
Stress	What happens beyond the expected load — and does it fail gracefully?
Spike	Can it absorb a sudden burst (a sale starts at noon)?
Endurance (soak)	Does it degrade over hours (memory leaks, growing tables)?
Scalability	Does adding resources raise capacity proportionally?

These need dedicated tools (Locust, JMeter, k6), many minutes per run, and an environment sized like production; they are usually run on a schedule rather than on every commit.

Real-World Case Study

HealthCare.gov, 2013. The U.S. federal health-insurance marketplace launched on October 1, 2013, and most visitors in its first weeks could not complete an enrollment. Testifying before the House Energy and Commerce Committee on October 24, representatives of two of its main contractors, CGI Federal and QSSI, said the integrated system had received only about two weeks of end-to-end testing before launch, and that months would have been appropriate[10]. Each component had been tested by its builder; the system as a whole, under real load, was tested last and least. System testing is where components that each "work" meet real users, real volume, and each other.

8.3 Security and Robustness

SYS-6 and SYS-7 are security requirements: access control, and safe handling of whatever arrives. System testers test them the way an attacker would: by sending what the documentation does *not* suggest.

```
HOSTILE_BODIES = [
    pytest.param({"sku": ["MUG"], "quantity": 1, "customer_email": "a@b.c"},
                 id="sku-is-a-list"),
    pytest.param({"sku": "MUG", "quantity": 10**30, "customer_email": "a@b.c"},
                 id="quantity-too-big-for-sqlite"),
    pytest.param({"sku": "MUG", "quantity": 101, "customer_email": "a@b.c"},
                 id="quantity-just-over-the-limit"),
    pytest.param({"sku": "MUG'; DROP TABLE products; --", "quantity": 1,
                  "customer_email": "a@b.c"}, id="sql-injection-in-sku"),
    # ... boolean quantity, object coupon, list body, 100 kB string body
]

@pytest.mark.parametrize("body", HOSTILE_BODIES)
def test_hostile_input_gets_a_json_4xx(shop, body):
    """SYS-7: rejected, in JSON, with no internals; and nothing changes."""
    response = shop.post("/orders", json=body)

    assert 400 <= response.status_code < 500, response.text[:200]
    assert response.headers["Content-Type"] == "application/json"
    assert "Traceback" not in response.text
    assert shop.stock("MUG") == 5          # the products table is intact
    assert shop.charges() == []
```

Finding F1. When this test was first run, against the shop with Week 10's input validation, two cases failed:

```
$ patch -p1 < specs/f1_week10_validation.patch      # Week 10's validation
$ pytest tests/system/test_robustness_and_security.py --tb=short
____________ test_hostile_input_gets_a_json_4xx[sku-is-a-list] ____________
E   AssertionError: <!doctype html>
E     <html lang=en>
E     <title>500 Internal Server Error</title>
E   assert 500 < 500
FAILED ...test_hostile_input_gets_a_json_4xx[sku-is-a-list]
FAILED ...test_hostile_input_gets_a_json_4xx[quantity-too-big-for-sqlite]
2 failed, 12 passed in 3.33s
```

This is a real defect, not a planted one: it is in Week 10's shop too. A JSON list where the API expects a string reached SQLite's parameter binding (`sqlite3.ProgrammingError`); a quantity of 10^{30} cannot be stored in a 64-bit integer (`OverflowError`). Neither exception was handled, so Flask answered with its default HTML error page — breaking the API contract's "never an HTML page", and announcing the framework to anyone probing the server. The tracebacks were in the server log in `tmp_path`, which is how the cause was found in minutes.

Notice, too, what passed. The SQL-injection attempt was refused as an unknown SKU and the `products` table survived, because every query in `repository.py` uses parameters (`?`), never string formatting — the defense OWASP recommends against injection, still among the most common web-application weaknesses[9].

Fixing F1 needed a requirement. Rejecting a list `sku` is a plain fix. But what is the largest valid quantity? The specification did not say, so the developer could not decide; the **product owner** did: at most 100 units per order. SYS-2 was updated, `validate_order()` now checks both fields, and a JSON handler for any `500` was added as defense in depth (SYS-7's "whatever the request contains"). The finding is recorded in the test plan's findings table with its severity and resolution.

Key Idea

System testing finds defects in *specifications* as well as code. A missing upper bound is not a bug a developer can fix alone: it is a missing requirement, and it goes back to whoever owns the requirements.

Access control (SYS-6) is a short parametrized test over every endpoint, sent with *no* key, plus one test that `/health` answers without one:

```
@pytest.mark.parametrize("method, path", [
    ("GET", "/products"), ("POST", "/orders"), ("GET", "/orders/1"),
])
def test_every_endpoint_except_health_needs_the_key(shop, method, path):
    response = requests.request(method, shop.url + path, timeout=5)  # no key
    assert response.status_code == 401
```

Common Pitfall

Testing access control only on the endpoints you remember. New endpoints are added later, and the access check is forgotten on one of them. Keep the list of endpoints in one place, and make adding a route without adding it to the list fail a test (Exercise 3).

8.4 Recoverability and Operability

A reliable system is not one that never meets a failure, but one that recovers from it. Three tests:

```
def test_orders_work_again_after_a_provider_outage(shop):
    """NFR-2: no restart needed once PayFast is back."""
    shop.provider(mode="unavailable")
    during = shop.order("MUG", 1)
    assert during.status_code == 503
    assert during.headers["Retry-After"] == "30"

    shop.provider(mode="normal")
    after = shop.order("MUG", 1)
    assert after.status_code == 201
    assert shop.stock("MUG") == 4              # only the second order took stock


def test_orders_and_stock_survive_a_restart(shop, start_shop):
    """SYS-8: stop the shop, start it again on the same database."""
    order = shop.order("MUG", 2).json()

    restarted = start_shop(restart_of=shop)

    assert restarted.get(f"/orders/{order['id']}").json() == order
```

```
    assert restarted.stock("MUG") == 3          # not re-seeded back to 5
```

The restart test exercises code that no lower-level test can reach: `serve.py`'s start-up path, which must create the schema *and* seed the catalog on a new database, but must *not* re-seed an existing one. A start-up script that re-seeded on every start would pass every unit and integration test — and silently reset the stock each time the shop was redeployed. The third test, `test_health_stays_ok_during_a_provider_outage`, checks a design decision recorded in NFR-3: a PayFast outage makes *orders* fail, not the shop, so a load balancer must not take the shop out of service (customers can still browse).

8.5 Other System Test Types

Type	What it checks
Usability	Can representative users complete tasks? Usually observed sessions, not automated tests.
Compatibility	Different browsers, operating systems, Python versions, database versions.
Installability	Does a clean install (or upgrade) work, and does uninstall leave nothing behind?
Configuration	Does each supported configuration (`--db`, `--port`, `--seed`) work?
Data migration	Do existing data survive an upgrade of the schema?
Accessibility	Can people using assistive technology use the user interface?

9 System Tests in the Suite

9.1 Markers and Run Time

Every system test module sets `pytestmark = pytest.mark.system` (registered in `pytest.ini`), so a level can be selected:

```
$ pytest -m system -q
23 passed, 35 deselected in 6.05s
```

Level	Tests	Time	Per test
Unit (`tests/unit`)	8	0.13 s	≈ 0.02 s
Integration (`tests/integration`)	13	0.5 s	≈ 0.04 s
System (`tests/system`)	23	6.0 s	≈ 0.26 s

Each system test starts a server, so it costs about ten times an integration test. That is why the test pyramid has few system tests at the top: they are the only tests that can find defects like the race and F1, and the most expensive to run and to diagnose. In a pipeline, a small **smoke** subset (start the system, `/health`, one main-success scenario) runs on every commit; the full system suite runs before a release.

9.2 One Defect, Four Levels

The demo's two deliberate defects, switched in by environment variables, give a compact picture of what each level can and cannot see:

	Unit	Integration	System	Acceptance
`SHOP_SERVICE=racy` (oversells)	pass	pass	**1 failed**	pass
`SHOP_COUPONS=per_unit` (companion reading)	pass	pass	pass	**2 failed**

No level replaces another. The second row is the subject of the companion reading: a system that passes all 23 system tests and is still not what the customer asked for.

10 Common System-Level Failures

Failure	Example in this reading	Found by	Could a lower level find it?
Concurrency defects	10 customers sold 5 mugs; stock left at 4	simultaneous orders against a threaded server	no: one request at a time in-process
Unhandled hostile input	list `sku`, huge `quantity` → HTML `500` (F1)	hostile-input system tests	yes, in principle — but nobody had thought to send it
Missing requirements	no maximum order quantity	the same tests, when fixing F1	no: it is a specification gap
Start-up and configuration	re-seeding an existing database on restart	restart test	no: `serve.py` is not under lower-level test
Dependency recovery	still failing after PayFast recovers	outage-then-recovery test	partly: integration tests saw one outage, not recovery
Operational behavior	health check failing during a PayFast outage	health test during an outage	no: it is a design decision about the whole system

The demo shop has the first two (one planted, one real) and was designed against the others; the tests in Section 8 keep it that way.

11 Putting It Together: A Hands-On Walkthrough

To plan system tests for a new feature, work through five steps. As an example, suppose the shop's staff need to restock: `POST /products/<sku>/restock` with `{"add": 20}`, authorized by a separate *staff* key.

Step 1 — Write or find the requirements. Functional: "Staff can add stock to a product; the catalog shows the new stock." Security: "Only the staff key may restock; the customer key gets `403`." And the one nobody writes down until asked: "A restock during a sale loses no units." Make each one measurable.

Step 2 — Update the test plan. Scope (a new endpoint, a new credential); environment (`serve.py` needs a staff key option); entry criteria (unit and integration tests for the new route pass); traceability rows for the new requirements.

Step 3 — Design the test cases, from the outside, for each test type:

ID	Type	Scenario	Expected (observable)
ST-R1	Functional	staff restocks TEE by 20	`200`; catalog shows TEE stock 20; a customer can now buy one
ST-R2	Security	restock with the customer key; with no key	`403`; `401`; stock unchanged
ST-R3	Security	`"add": -5`, `"add": "20"`, `"add": 10**30`	JSON `400`; stock unchanged (remember F1)
ST-R4	Concurrency	restock MUG by 10 while 10 customers each buy one	stock ends at $5+10-$(mugs sold); no unit lost
ST-R5	Recoverability	restock, then restart the shop	stock still includes the restock

ST-R4 is the one to worry about. `OrderRepository` already has a `set_stock()` method — the one the racy service misused. A restock written as "read the stock, add 20, `set_stock()`" would lose units to concurrent sales in exactly the way Figure 2 shows.

Step 4 — Prepare the environment. Add the staff key to `serve.py`'s options and to `RunningShop`; reuse `order_all_at_once` for ST-R4 with a provider delay to force the overlap.

Step 5 — Automate and trace. Add the tests to `tests/system/` with the `system` marker, put the requirement IDs in their docstrings, and add the rows to the plan's traceability table. Exercise 6 asks you to do this.

12 Best Practices

1. **Start the real system,** the way it is deployed, with a fresh environment per test.
2. **Test only through external interfaces;** control only the test environment.
3. **Make quality requirements measurable** before testing them: a load, a condition, a threshold.
4. **Test every kind of user:** customers in parallel, operators restarting, attackers probing.
5. **Force the conditions you are testing** — barriers and delays for races, controllable fakes for outages — so results do not depend on luck.
6. **Keep the server's log** with every test, and read it first when a test fails.
7. **Trace every requirement to a test,** and every finding to a fix or a recorded decision.
8. **Keep system tests few and meaningful,** with a smoke subset on every commit and the full set before release.

13 Common Mistakes

Common Pitfall

Calling in-process tests system tests. Flask's test client, a mocked database, or a single thread means some part of the system is not running as deployed. Name the level honestly; it tells the reader what the test can and cannot find.

Common Pitfall

Reaching inside the system to check results. Observe through external interfaces; control only the test environment.

Common Pitfall

Testing only the happy path, one customer at a time. The shop's most expensive defect (the race) appears only under concurrency; its security defect (F1) only under hostile input.

Common Pitfall

Unmeasurable quality requirements. "Fast" and "secure" cannot fail a test. Ask for a load, a condition, and a threshold.

Common Pitfall

Timing tests with no margin. A latency assertion set at last week's measurement fails on a busy machine. Set budgets from the requirement, with headroom, and run timing-sensitive tests repeatedly before trusting them.

Common Pitfall

Starting system testing before integration testing is done. System test failures that are really integration defects are slow to diagnose: the failure says *that*, not *where* (Week 10's `assert 502 == 201`).

14 Applying This to Your Semester Project

- Write a short system requirements list for your project, with IDs, and at least three measurable non-functional requirements (performance, security, recoverability, or another ISO/IEC 25010 characteristic).
- Write a one-page system test plan (Section 5). Put it in the repository and keep its traceability table current.
- Build a harness that runs your system as it is run for real: a server process, a command-line program invoked with `subprocess`, or a packaged installation. Put the tests in `tests/system/` with a `system` marker.
- Include at least one test each for hostile input, a failing dependency, and (if your system serves more than one user) concurrent use.
- In your report, list each system-level finding with its severity, the test that found it, and its resolution — including requirements you had to add.
- For Track A (existing open-source) projects: run the project as its users do (its CLI, its documented entry point) before writing any system test; its documentation is your system test basis.

15 A Practical Mental Model

When you write a system test, think:

> "Would this test still work if the system were rewritten in another language? If not, it is looking inside."

When you read a requirement, think:

> "Who else touches the system — other customers at the same time, an operator restarting it, an attacker — and what does the requirement promise each of them?"

When a system test fails, think:

> "Server log first. Then: is the code wrong, or is the requirement missing?"

16 Summary

Test type	**Shop example**	**Found**
Functional scenarios	browse, order, look up; alternative flows	—
Concurrency	10 customers, 5 mugs	the racy service's oversell
Performance	10 customers within 1 s	—
Security / robustness	hostile bodies; no-key requests	F1: HTML `500` pages
Recoverability	outage then recovery; restart	—

Term	**Meaning**
System testing	Testing the whole, deployed system against its specification
System test basis	System requirements, use cases, quality requirements
Non-functional requirement	A measurable quality: performance, security, reliability, ...
Test environment	Everything around the system under test, as close to production as practical
Entry / exit criteria	When system testing may start; when it is done
Lost update	Two writers each overwrite the other's change; caused here by check-then-act
Smoke test	A small, fast subset that shows the system starts and its main path works

Key Idea

Test the system as it runs, from where its users stand. System testing is the first level at which the software is started, shared, attacked, and restarted the way it will be in production — and the first at which defects that live in those conditions can be found.

17 Practice Exercises

Exercise 1 — A Scenario

Write a system test for this alternative flow: a customer orders 5 mugs (all of them), then a second customer's order for 1 mug is refused, then the first customer's order is looked up. Trace it to requirements.

Exercise 2 — A Concurrency Test That Passes by Luck

Remove the barrier and set the provider delay to 0 in `test_ten_customers_race_for_the_last_five_mugs`. Run it ten times with `SHOP_SERVICE=racy`. How often does it fail? Explain the result in terms of Figure 2.

Exercise 3 — Access Control That Cannot Forget

Rewrite `test_every_endpoint_except_health_needs_the_key` so that it discovers the routes from the running system instead of a hand-written list. (Hint: what could the shop expose to make this possible, and should it be exposed in production?) Then add a route `GET /orders` without the key check and show your test catches it.

Exercise 4 — A New Quality Requirement

Write a measurable requirement for the case where PayFast takes 10 seconds to answer, and a system test for it. (Look at `PaymentClient`'s timeout first.) What response should the customer get, and how long should it take?

Exercise 5 — A Test Plan

Write a one-page system test plan for your own course project, with the sections of Section 5. Include at least two non-functional requirements, each with a load or condition and a threshold.

Exercise 6 — Restocking

Implement the restock feature of Section 11 and its five system tests, ST-R1 to ST-R5. First write ST-R4 against a "read, add, `set_stock()`" implementation and show that it fails; then fix the implementation.

18 Discussion Questions

1. Week 10's `test_api.py` sent HTTP requests to the fully assembled shop. Why are those tests integration tests and not system tests?
2. Why should a system test not read the database directly? When might it be acceptable, and what would you call such a test?
3. The fake PayFast's `/_test/` endpoints are not part of the system under test. What would go wrong if the *shop* had such endpoints in production?
4. Why could no integration test in `tests/integration/` find the racy service's defect, however it was written?

5. Who should decide the maximum order quantity, and why not the developer who fixed F1?
6. What makes NFR-1 testable, and what would a real load test need that ours lacks?
7. Should `/health` report unhealthy during a PayFast outage? Argue both sides.
8. Why does the test pyramid have the fewest tests at the system level, even though system tests find defects no other level can?

19 Key Takeaways for SE 413/513

1. **System testing tests the whole system, as deployed, against its specification** — from the outside, through its external interfaces.
2. **Its test basis includes non-functional requirements,** which must be measurable to be testable.
3. **Plan it:** scope, environment, approach, entry and exit criteria, traceability, and risks.
4. **Start the real system;** observe through its interfaces; control only the test environment.
5. **Test what production will do to it:** many users at once, hostile input, failing dependencies, restarts.
6. **Some defects live only there:** the race and F1 passed every unit and integration test.
7. **System tests find missing requirements,** which go back to the people who own the requirements.

20 References

- Test levels, system testing, test planning, and test types.[2, 3, 4, 5]
- Product quality characteristics.[6]
- Test documentation and test plans.[7]
- Performance testing.[8]
- Injection and other web-application security risks.[9]
- The HealthCare.gov launch.[10]
- Using `pytest`, this course's default testing framework.[11]
- Course material on software testing principles and related testing practices.[1]

References

[1] Hasan, K. (2026). *Software Testing*. Grand Valley State University. https://github.com/mdkamrulhasan/software-testing

[2] International Software Testing Qualifications Board. (2024). *Certified Tester Foundation Level (CTFL) Syllabus*, v4.0.1.

[3] Jorgensen, P. C. (2013). *Software Testing: A Craftsman's Approach* (4th ed.). CRC Press.

[4] Ammann, P., & Offutt, J. (2016). *Introduction to Software Testing* (2nd ed.). Cambridge University Press.

[5] Aniche, M. (2022). *Effective Software Testing: A Developer's Guide*. Manning Publications.

[6] ISO/IEC. (2023). *ISO/IEC 25010:2023 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model.*

[7] ISO/IEC/IEEE. (2021). *ISO/IEC/IEEE 29119-3:2021 Software and systems engineering — Software testing — Part 3: Test documentation.*

[8] Molyneaux, I. (2014). *The Art of Application Performance Testing* (2nd ed.). O'Reilly Media.

[9] OWASP Foundation. (2021). *OWASP Top 10:2021*. https://owasp.org/Top10/

[10] Nextgov. (2013, October 24). Officials spent just two weeks testing HealthCare.gov prior to launching it. https://www.nextgov.com/cxo-briefing/2013/10/officials-spent-just-two-weeks-testing-healthcaregov-prior-launching-it/72608/

[11] Okken, B. (2022). *Python Testing with pytest* (2nd ed.). Pragmatic Bookshelf.